



RV College of
Engineering®

Department of Computer Science & Engineering

Distributed
Computing



Parallel
Computing

PADP 21CS72

Dr. Minal Moharir

Go, change the world®



RV College of
Engineering®

Objective : Motivation to learn Parallel Computing



Go, change the world®



RV College of
Engineering®

Serial Computing



If **1 Man** takes **10 hours** to complete the job; then
How long will it take if **10 Men** work **together**

Go, change the world®



RV College of
Engineering®

Parallel Computing

THINK
PARALLEL

PARALLEL Computing!

सी डैक
CDAC



- Save wall clock time
- Solve larger problems
- Provide concurrency
 - (do multiple things at the same time)

Parallelism is the future of computing!

Go, change the world®

Applications

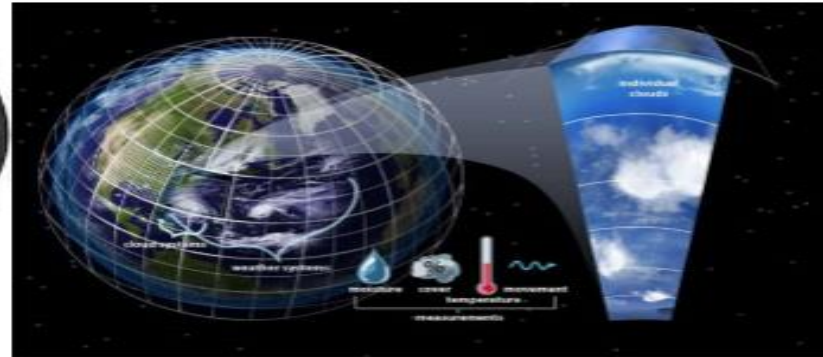
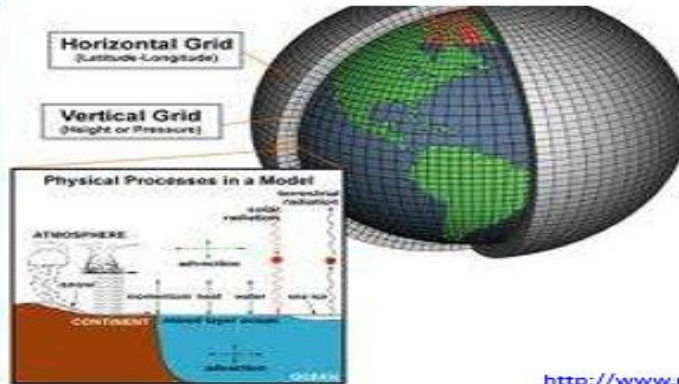
THINK
PARALLEL

Application Drivers for HPC

सी डैक
CDAC

Weather Forecasting

- Atmosphere modeled by dividing it into 3-dimensional cells.
- Calculations of each cell repeated many times to model passage of time.



http://www.nsf.gov/news/special_reports/clouds/images/nsf_clouds_computing.pdf



Applications



Weather Forecasting Example



- Consider atmosphere to be divided into cells of size 1 mile $\times$ 1 mile $\times$ 1 mile to a height of 10 miles
- Global region can be schemed as 2×10^9 cells.
- Each cell has many physical variables (temperature, pressure, humidity, wind speed and direction, etc) to be computed
- Suppose each calculation requires 200 floating point operations. In *one time step*, 10^{11} floating point operations necessary.
- To forecast the weather over 7 days using 1-minute intervals, takes $(7 * 24 * 60 * 10^{11}) = 10080 * 10^{11} = 10^{15}$ FP operations
- a computer operating at 1Gflops (10^9 floating point operations/s) takes 10^6 seconds or over 10 days.
- To perform calculation in 5 minutes requires computer operating at 3.4 Tflops (3.4×10^{12} floating point operations/sec).



Go, change the world®



UNITS



Units of Measurement in HPC



- **Mflop/s** 10^6 flop/sec
- **Gflop/s** 10^9 flop/sec
- **Tflop/s** 10^{12} flop/sec
- **Pflop/s** 10^{15} flop/sec
- **Mbyte** 10^6 byte (also $2^{20} = 1048576$)
- **Gbyte** 10^9 byte (also $2^{30} = 1073741824$)
- **Tbyte** 10^{12} byte
(also $2^{40} = 10995211627776$)
- **Pbyte** 10^{15} byte
(also $2^{50} = 1125899906842624$)

Go, change the world®

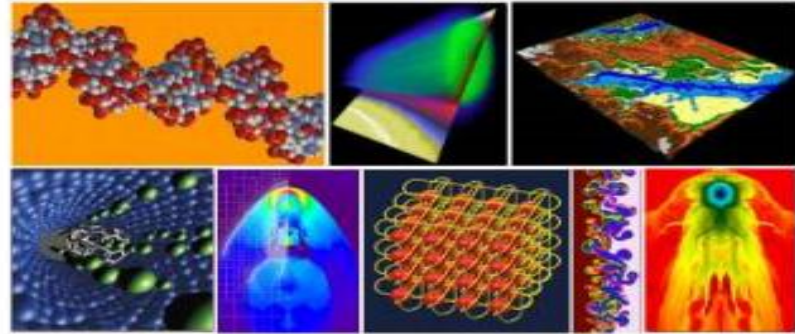


RV College of
Engineering®

Applications

THINK PARALLEL Applications of Parallel Computing सी डैक CDAC

- Prediction of weather, climate, global changes
- Modeling motion of astronomical bodies
- Human genome
- Challenges in materials sciences
- Semiconductor design
- Superconductivity
- Structural biology
- Design of drugs
- Challenges in transportation
- Vehicle dynamics
- Nuclear fusion
- Enhanced oil and gas recovery
- Computational ocean sciences
- Speech
- Vision
- Visualization and graphics



Go, change the world®



RV College of Engineering®

Where are we Today?

Computer Technology Today have made incredible progress in roughly 65-years after first general purpose electronic computer was created.



Go, change the world[®]



RV College of
Engineering®

Where are we Today?

THINK PARALLEL **Computer Technology Today** **सी डैक CDAC**

Today's Desk Top Computer for US\$ 500 more powerful than a

Powerful : Performance, Main Memory & Disk Storage

High Performance Computing (HPC) System two decades ago for US\$ 1 Million

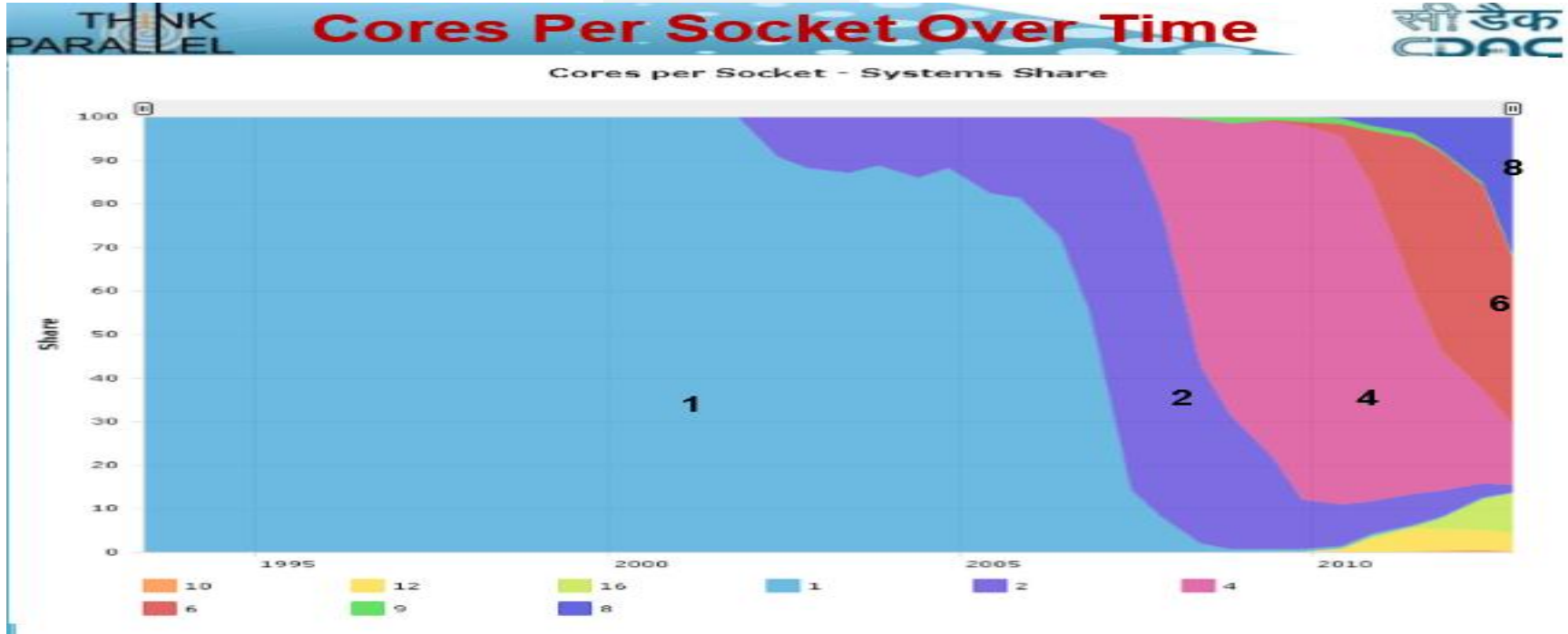
A photograph of a large, industrial-scale server room. The room is filled with numerous rows of black server racks, each with multiple bays. The racks are arranged in a grid-like pattern, extending into the distance. The floor is made of light-colored tiles, and the walls are white. The overall impression is one of a massive, powerful computing system.

Go, change the world®



RV College of
Engineering®

Where are we Today?



Go, change the world®



Definition

THINK
PARALLEL

High Performance Computing

सी डैक
CDAC

- **Definition 1 (Wikipedia)**
 - High Performance Computing (HPC) uses Supercomputers and Computer Clusters to solve advanced computational problems. Today, Computer Systems approaching the Teraflops- region are counted as HPC-Computers
- **Definition 2**
 - High Performance Computing (HPC) is the use of Parallel Processing for running advanced application programs efficiently, reliably and quickly. The term applies especially to systems that function above a TeraFlop or 10^{12} Floating-Point Operations Per Second.

Go, change the world®



RV College of
Engineering®

Benchmarking

THINK
PARALLEL

TOP500

सी डेक
CDAC

Jack Dongarra, H. Simon, E. Strohmaier and H. Meuer

- Listing of the 500 most powerful Supercomputers in the World
- Based on **LINPACK Benchmark** : Measure of System's **Floating Point Computing Power**
- How **fast** a Computer solves a Dense N by N System of Linear Equations

$$Ax=b, \text{ dense problem}$$



- Result is reported as **Rate of Execution in MFlops /Sec**
- Updated **twice a year**
- Supercomputing Conference (SC) in United States in November**
- International Supercomputing Conference (ISC) in Europe in June**

Go, change the world®



RV College of
Engineering®

www.top500.org June 2024

Rank	System	Cores	Rmax (PFlop/s)	Rpeak (PFlop/s)	Power (kW)
1	Frontier - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE DOE/SC/Oak Ridge National Laboratory United States	8,699,904	1,206.00	1,714.81	22,786
2	Aurora - HPE Cray EX - Intel Exascale Compute Blade, Xeon CPU Max 9470 52C 2.4GHz, Intel Data Center GPU Max, Slingshot-11, Intel DOE/SC/Argonne National Laboratory United States	9,264,128	1,012.00	1,980.01	38,698
3	Eagle - Microsoft NDv5, Xeon Platinum 8480C 48C 2GHz, NVIDIA H100, NVIDIA Infiniband NDR, Microsoft Azure Microsoft Azure United States	2,073,600	561.20	846.84	
4	Supercomputer Fugaku - Supercomputer Fugaku, A64FX 48C 2.2GHz, Tofu interconnect D, Fujitsu RIKEN Center for Computational Science Japan	7,630,848	442.01	537.21	29,899
5	LUMI - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE EuroHPC/CSC Finland	2,752,704	379.70	531.51	7,107
6	Alps - HPE Cray EX254n, NVIDIA Grace 72C 3.1GHz, NVIDIA GH200 Superchip, Slingshot-11, HPE	1,305,600	270.00	353.75	5,194

Go, change the world®



RV College of
Engineering®

Principles of CA Design



Take Advantage of Parallelism

e.g. multiple processors, disks, memory banks, pipelining, multiple functional units

Principle of Locality

Reuse of data and instructions

Temporal locality states that recently accessed items are likely to be accessed in the near future.

Spatial locality says that items whose addresses are near one another tend to be referenced close together in time.

Focus on the Common Case

The instruction fetch and decode unit of a processor may be used much more frequently than a multiplier, so optimize it first

Addition of 2-nos., Overflow

Go, change the world®



RV College of
Engineering®

Dependency Analysis

- **Their order of execution must not matter!**

- **Example 1**

a=1;

b=2;

Example 2

a=2;

b=a;

Go, change the world®

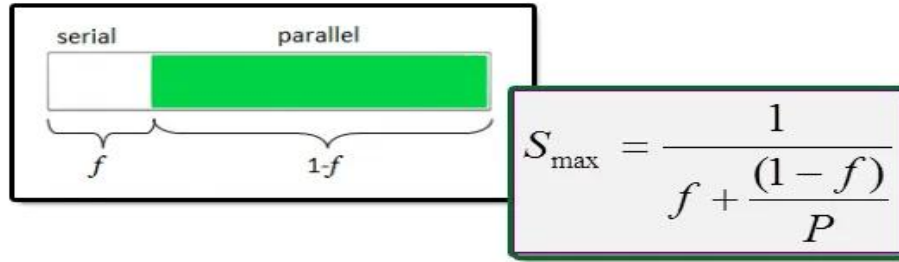


RV College of
Engineering®

Dependency Analysis

Inherently serial: I/O

Speedup



Go, change the world®



RV College of
Engineering®

Program Programming Paradigm

OpenMp

MPI

CUDA

OpenACC

OpenCL

Go, change the world®



RV College of
Engineering®

What is OpenMP?

De-facto standard API for writing shared memory parallel applications in C, C++, and Fortran

Consists of:

1. Compiler Directives
2. Run time Libraries
3. Environment Variables

Specifications are mentationed by OpenMp Architecture Review Board(<http://www.openmp.org>)

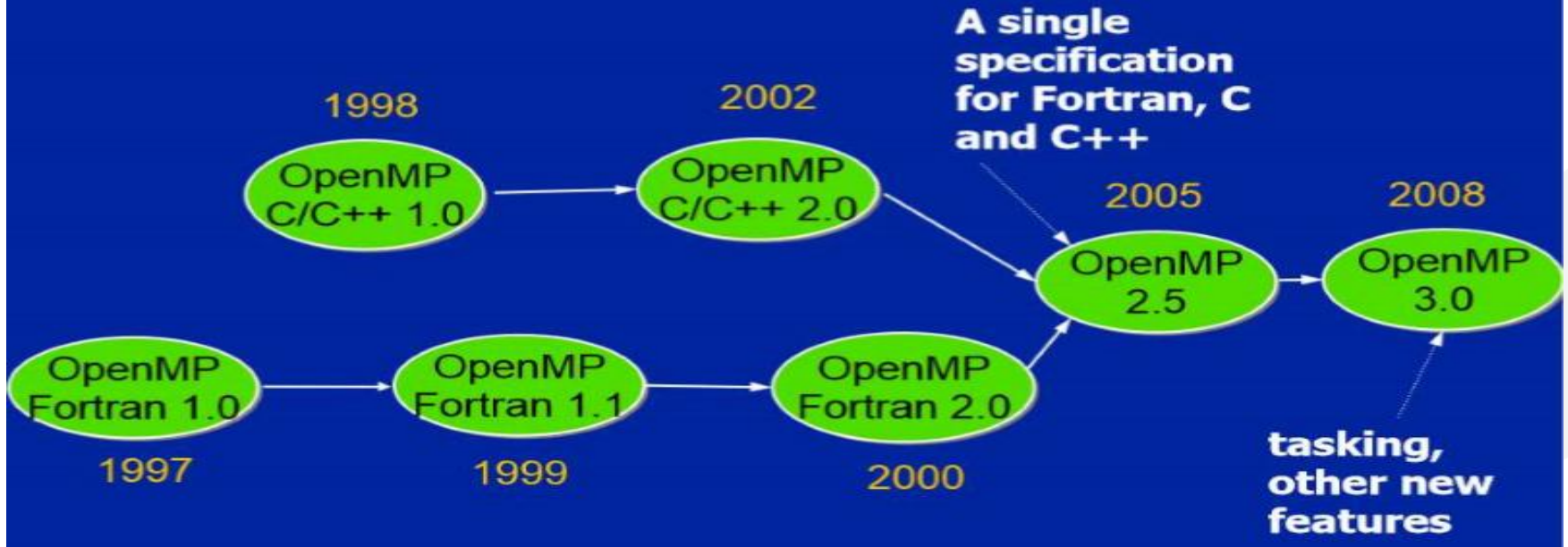
Version 4.5 has been released by in Nov. 2015

Go, change the world®



OpenMP?

OpenMP Release History



Go, change the world®



RV College of
Engineering®

When to Consider

When Compiler cannot find parallelism

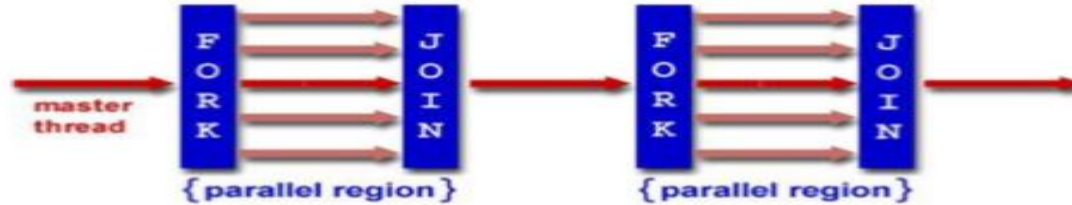
Use explicit PARALLELIZATION- OpenMp

Go, change the world®



Memory Model

- **Shared Memory, Thread Based Parallelism**
- **Explicit Parallelism**
- **Fork - Join Model**

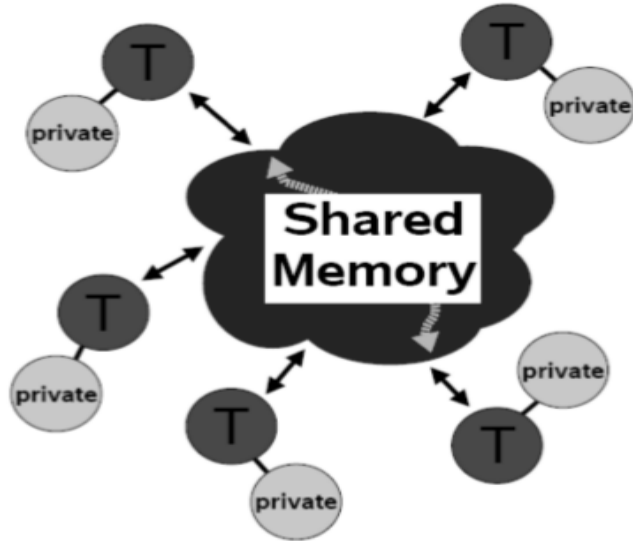


- **Compiler Directive Based**
- **Nested Parallelism Support**
- **Dynamic Thread**
- **Memory Model : Flush often**

Go, change the world®



Memory Model



- Data is **private** or **shared**.
- All threads have **access** to **same globally shared memory**.
- Shared data accessible by all threads.
- Private accessed only **by owned threads**.
- Data transfer is transparent to programmer.
- **Synchronization** takes place, but it is almost **implicit**.

Go, change the world®



Compilation

GNU	gcc	-fopenmp
Linux Opteron/Xeon	g++	
IBM Blue Gene	g77	
	gfortran	

GNU Compiler Example :

- **gcc -o omp_hello -fopenmp omp_hello.c**

Go, change the world®



RV College of
Engineering®

Advantages of OpenMP

- **Good performance and scalability**
 - ✓ If you do it right
- **De-facto and mature standard**
- **An OpenMP program is portable**
 - ✓ Supported by a large number of compilers
- **Requires little programming effort**

Go, change the world®



Components of OpenMP

Compiler Directives

- **Parallel Construct**
- **Work Sharing**
- **Synchronization**
- **Data Environment**
 - ✓ private
 - ✓ first private
 - ✓ last private
 - ✓ shared
 - ✓ reduction

Environment Variables

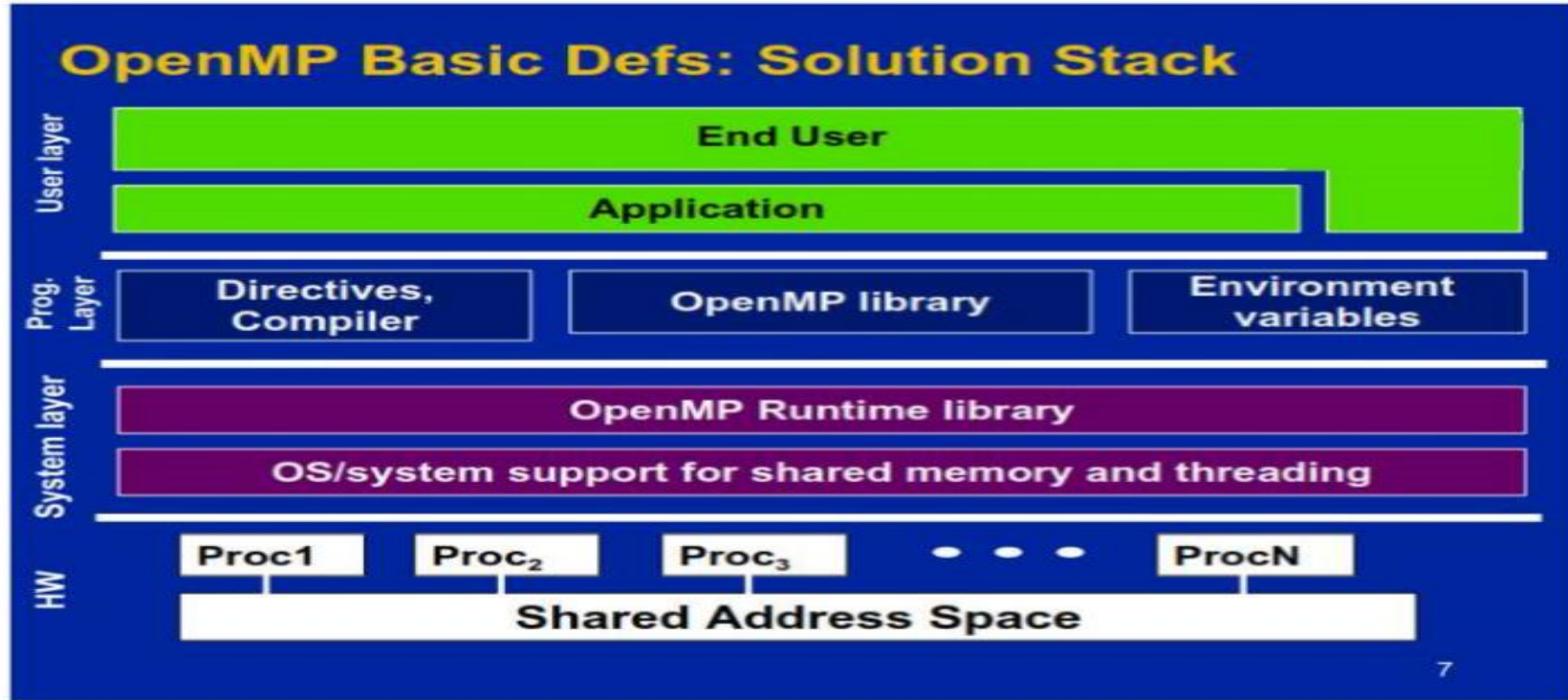
- **Number of threads**
- **Scheduling Type**
- **Nested parallelism**
- **Dynamic Thread**
- **Adjustment**

Runtime Library routines

- **Number of threads**
- **Thread ID**
- **Dynamic thread**
- **adjustment**
- **Nested parallelism**



OpenMP Stack



Go, change the world®



OpenMp Directives

❖ **#pragma omp directive-name [clause, clause..] new-line**

- **Eg: #pragma omp parallel default(shared) private(beta,pi)**

❖ **General Rules:**

- **Case sensitive**
- **Compiler Directives follow C/C++ standards**
- **Only one directive-name to be specified per directive**
- **Each directive applies to at most one succeeding statement.**
- **Use ("\\") for continuing on succeeding lines.**

Go, change the world®



Restrictions

- A parallel region must be a **structured block** that does not span multiple routines or code files
- It is **illegal to branch into or out** of a parallel region
- Only a **single IF clause** is permitted
- Only a **single NUM_THREADS** clause is permitted



How many Threads?

- **The number of threads in a parallel region is determined by the following factors, in order of precedence:**
 - ✓ **Evaluation of the IF clause**
 - ✓ **Setting of the NUM_THREADS clause**
 - ✓ **Use of the omp_set_num_threads() library function**
 - ✓ **Setting of the OMP_NUM_THREADS environment variable**
 - ✓ **Implementation default - usually the number of CPUs on a node, though it could be dynamic (see next bullet).**
- **Threads are numbered from 0 (master thread) to N-1**



Data Scoping

The OpenMP Data Scope Attribute Clauses are used to explicitly define how variables should be scoped. They include:

- ✓ PRIVATE
- ✓ FIRSTPRIVATE
- ✓ LASTPRIVATE
- ✓ SHARED
- ✓ DEFAULT
- ✓ REDUCTION

Data Scope Attribute Clauses are used in conjunction with several directives (**PARALLEL**, **DO/for**, and **SECTIONS**) to control the scoping of enclosed variables

Go, change the world[®]



Data Scoping

❖ private clause

- **This declares variables in its list to be private to each thread**
- **Format**

✓ **private (list)**

```
Eg:   int B = 10;  
       #pragma omp parallel private(B)  
       B = ... ;
```

- A private **un-initialised copy** of B is created before the parallel region begins
- **B value is not the same** within the parallel region as outside

Go, change the world[®]



Data Scoping

❖ firstprivate clause

- **Format**

- ✓ first private (list)

- ✓ Eg: `int B;`

- `B = 10;`

- `#pragma omp parallel firstprivate(B)`

- `B = B + ... ;`

- A private **initialized copy** of B is created before the parallel region begins
- The copy of each thread **gets the same value**



Data Scoping

❖ SHARED Clause

- ✓ A shared variable **exists in only one memory location** and all threads can read or write to that address

- **Format**

- ✓ shared (list)

❖ DEFAULT Clause

- ✓ Specify default scope for all variables in the lexical extent.

- **Format**

- ✓ default (shared | none)

❖ LASTPRIVATE Clause

- ✓ Value from the **last** loop iteration **assigned the original variable** object.

Go, change the world®



Reduction Clause

✓ **Variables which needed to be shared & modified by all the processors**

- **Format**

- ✓ reduction (operator: list)

- **Example**

```
total = 0.0;
# pragma omp parallel for private ( i, p ) /
  shared ( n, x ) reduction ( +: total )

for ( i = 0; i < n; i++ )
{
    p = ( ( x[i] - 7 ) * x[i] + 4 ) * x[i] - 83;
    total = total + p;
}
```

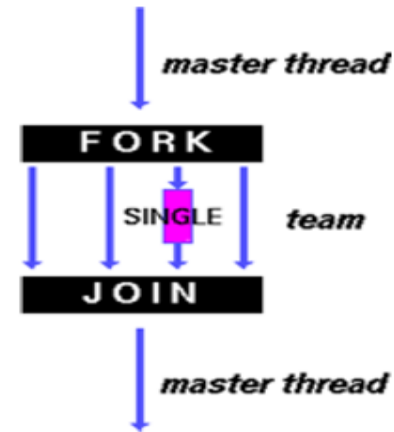
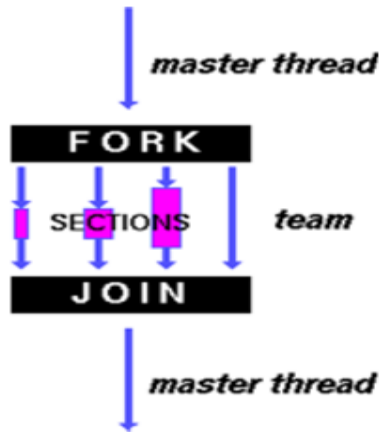
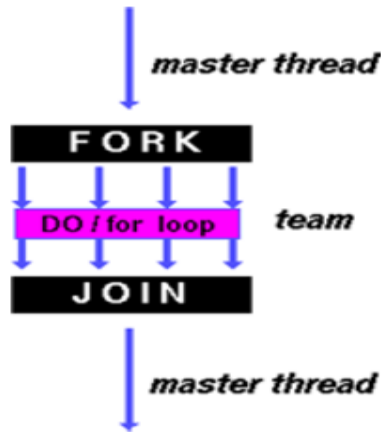
Symbol	Meaning
+	Summation
-	Subtraction
*	Product
&	Bitwise AND
	Bitwise OR
^	shift
&&	Logical AND
	Logical OR

Go, change the world®



Reduction Clause

- **FOR** - data parallelism
- **SECTIONS** - functional parallelism
- **SINGLE** - serializes a section of code



Go, change the world®



MPI: Message Passing Interface

- **A message passing library specification**
- **Message passing among processes in parallel computing**
- **Meant for clusters and network of workstations**
- **Message Passing Standard for Parallel Programs**
 - MPI Implementation left to individual vendors.
 - Most commonly supports C, Fortran, C++ programs



RV College of
Engineering®

MPI: Message Passing Interface

OpenMPI Implementation <https://www.open-mpi.org/>

- ▶ Open source (Git)
- ▶ Collaborators (partial)
 - ▶ Auburn, Wisconsin at La Crosse, Michigan
 - ▶ Los Alamos National Lab, Oak Ridge National Lab, Sandia National Lab
 - ▶ Amazon, AMD, ARM, Broadcom, Cisco, Facebook, Fujitsu, IBM, Intel, nVIDIA, Oracle

Go, change the world®



MPI: Message Passing Interface

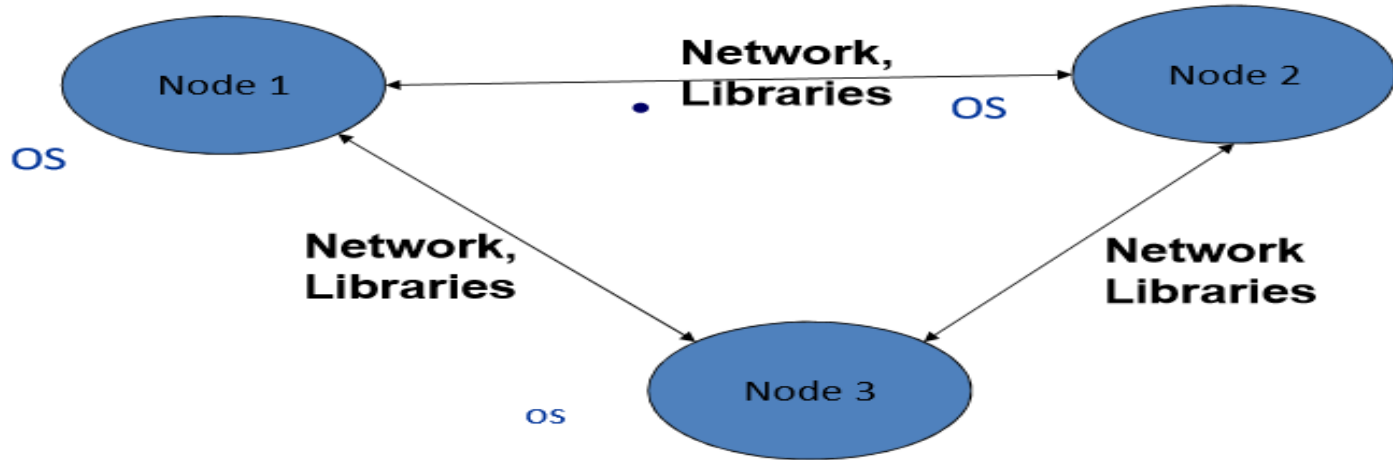
MPICH Implementation <http://www.mpich.org/>

- ▶ Open source (Git)
- ▶ 'CH' from *Chameleon* portability system
- ▶ MPICH1 (1992), MPICH2 (2001), MPICH3 (2012)
- ▶ Collaborators (partial)
 - ▶ University of British Columbia, Ohio State
 - ▶ Microsoft, IBM, Cray

Go, change the world®



MPI: Message Passing Interface



- Multiple Nodes connected together to form a Computer Cluster

Go, change the world®



RV College of
Engineering®

MPI: Message Passing Interface



Go, change the world®



RV College of
Engineering®

Program Programming Paradigm

```
#include <stdio.h>
#include <mpi.h>
main(int argc, char **argv)
{   int ierr;
    ierr = MPI_Init(&argc, &argv);
    printf("Hello world\n");
    ierr = MPI_Finalize();
}
```

mpicc hello.c -o hello
mpirun -np 4 hello

Go, change the world®



RV College of
Engineering®

OpenMP

```
#include <omp.h>
#include <stdio.h>
int main (int argc, char
*argv[])
{
    #pragma omp parallel
        printf("Hello World");
}
```

Compile:
gcc –
fopenmp
hello.c

Go, change the world®



RV College of Engineering®

OpenMP

```
#include <omp.h>
#include <stdio.h>
int main (int argc, char
*argv[])
{
#pragma omp parallel
num_threads(8)
    printf("Hello World");
}
```

Go, change the

Go, change the world[®]

Compile:

```
gcc -fopenmp hello.c
```



RV College of Engineering®

OpenMP

```
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>

int main (int argc, char *argv[])
{
    int nthreads, tid;

    /* Fork a team of threads giving them their own copies of variables */
    #pragma omp parallel private(nthreads, tid)
    {

        /* Obtain thread number */
        tid = omp_get_thread_num();
        printf("Hello World from thread = %d\n", tid);

        /* Only master thread does this */
        if (tid == 0)
        {
            nthreads = omp_get_num_threads();
            printf("Number of threads = %d\n", nthreads);
        }

        /* All threads join master thread and disband */
    }
}
```

Compile:

```
gcc -fopenmp hello.c
```

Go, change the world[®]



RV College of
Engineering®

```
#include <stdio.h>
#include <mpi.h>
main(int argc, char **argv)
{
    int ierr, num_procs, my_id;
    ierr = MPI_Init(&argc, &argv);
    /* find out MY process ID, and how many processes were started. */
    ierr = MPI_Comm_rank(MPI_COMM_WORLD, &my_id);
    ierr = MPI_Comm_size(MPI_COMM_WORLD, &num_procs);
```

Program Programming Paradigm

```
printf("Hello world! I'm process %i out of %i processes\n",
      my_id, num_procs);

ierr = MPI_Finalize();
}
```

Go, change the world®



RV College of
Engineering®

```
#include <stdio.h>
#include <mpi.h>
main(int argc, char **argv)
{
    int ierr, num_procs, my_id;
    ierr = MPI_Init(&argc, &argv);
    /* find out MY process ID, and how many processes were started. */
    ierr = MPI_Comm_rank(MPI_COMM_WORLD, &my_id);
    ierr = MPI_Comm_size(MPI_COMM_WORLD, &num_procs);
```

Program Programming Paradigm

```
printf("Hello world! I'm process %i out of %i processes\n",
      my_id, num_procs);

    ierr = MPI_Finalize();
}
```

Go, change the world®



RV College of
Engineering®



THANK YOU

Go, change the world®